NOTE À MOI-MÊME VA **ENREGISTRER** LA RENCONTRE MAINTENANT

(TU VAS OUBLIER SINON)

UNIVERSITÉ LAVAL

DIVISION AVIONIQUE

Programmation embarquée

Présenté par Samuel Gagnon

REV A
Novembre 2025

UNIVERSITÉ LAVAL

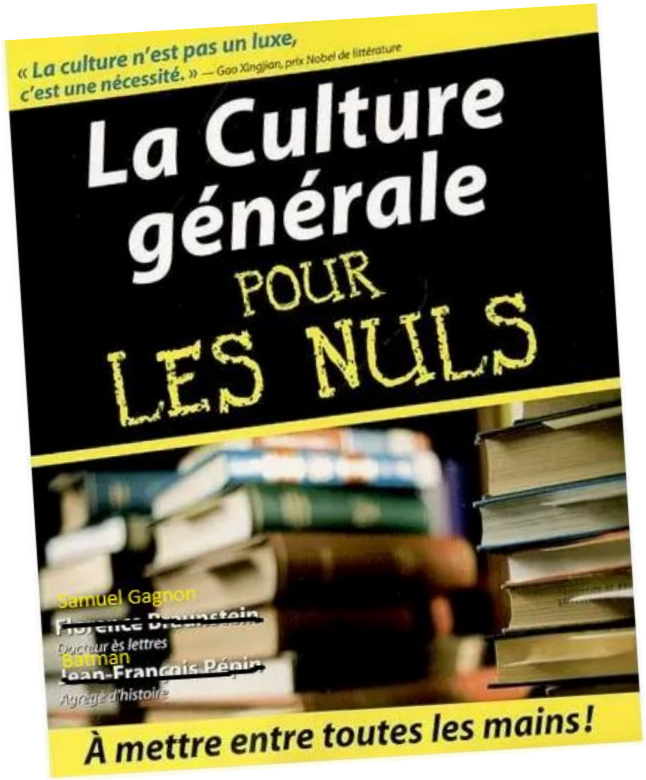
Table des matières

- Concepts généraux
- Programmation embarquée en C
- ODB
- Mini projet

UNIVERSITÉ LAVAL

CONCEPTS GÉNÉRAUX

(ÇA RESTE UN ATELIER, NON?)



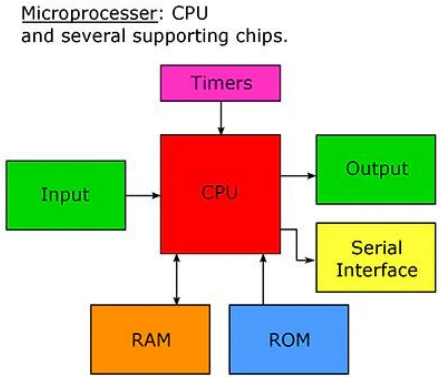
Concepts généraux

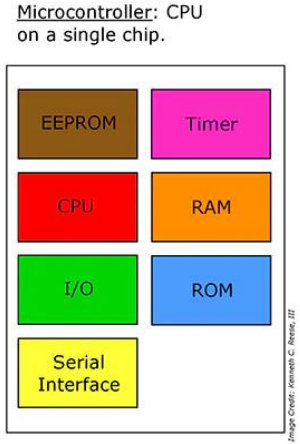
À quoi ça sert un MCU?

- MCU vs CPU
- Datasheet
- Horloges (crystal interne/externe, PLL)
- GPIOs
- TIMER & PWM
- ADC / DAC
- Protocoles de communication
- Caractéristiques des capteurs

MCU vs CPU

Et oui, y'a une différence …





- MCU

« petit ordinateur intégré (CPU + mémoire + IOs), faible consommation, pour systèmes embarqués »

- CPU

« processeur puissant, dépendant de composants externes, pour ordinateurs ou systèmes complexes »

Wink-Wink, un STM32/Arduino, c'est un MCU !!!

Datasheet

Pas encore de la documentation …

- Fonctionnalités
- Pinout
- Caractéristiques
- Schémas d'applications typiques
- Arbres d'architectures
- Recommandations externes

life.augmented

STM32F103x8
STM32F103xB

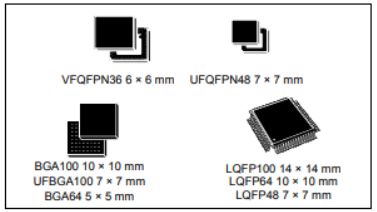
Medium-density performance line Arm®-based 32-bit MCU with 64 or 128 KB Flash, USB, CAN, 7 timers, 2 ADCs, 9 com. interfaces

Datasheet - production data

Features

Includes ST state-of-the-art patented technology

- Arm® 32-bit Cortex®-M3 CPU core
  - 72 MHz maximum frequency, 1.25 DMIPS/MHz (Dhrystone 2.1) performance at 0 wait state memory access
  - Single-cycle multiplication and hardware division
- Memories
  - 64 or 128 Kbytes of Flash memory
  - 20 Kbytes of SRAM
- Clock, reset and supply management
  - 2.0 to 3.6 V application supply and I/Os
  - POR, PDR, and programmable voltage detector (PVD)
  - 4 to 16 MHz crystal oscillator
  - Internal 8 MHz factory-trimmed RC
  - Internal 40 kHz RC
  - PLL for CPU clock
  - 32 kHz oscillator for RTC with calibration
- Low-power
  - Sleep, Stop and Standby modes
  - V_{BAT} supply for RTC and backup registers
- 2x 12-bit, 1 µs A/D converters (up to 16 channels)
  - Conversion range: 0 to 3.6 V
  - Dual-sample and hold capability
  - Temperature sensor
- DMA
  - 7-channel DMA controller
  - Peripherals supported: timers, ADC, SPIs, I^2Cs and USARTs
- Up to 80 fast I/O ports
  - 26/37/51/80 I/Os, all mappable on 16 external interrupt vectors and almost all 5 V-tolerant



- Debug mode:
  - Serial wire debug (SWD) and JTAG interfaces
- Seven timers
  - Three 16-bit timers, each with up to 4 IC/OC/PWM or pulse counter and quadrature (incremental) encoder input
  - 16-bit, motor control PWM timer with dead-time generation and emergency stop
  - Two watchdog timers (independent and window)
  - SysTick timer 24-bit downcounter
- Up to nine communication interfaces
  - Up to two I^2C interfaces (SMBus/PMBus®)
  - Up to three USARTs (ISO 7816 interface, LIN, IrDA capability, modem control)
  - Up to two SPIs (18 Mbit/s)
  - CAN interface (2.0B Active)
  - USB 2.0 full-speed interface
- CRC calculation unit, 96-bit unique ID
- Packages are ECOPACK®

Table 1. Device summary

Reference	Part number
STM32F103x8	STM32F103C8, STM32F103R8 STM32F103V8, STM32F103T8
STM32F103xB	STM32F103RB STM32F103VB, STM32F103CB, STM32F103TB

Fréquence coeur

Flash & Alimentation

ADCs

IOs

SWD & JTAG

Timers

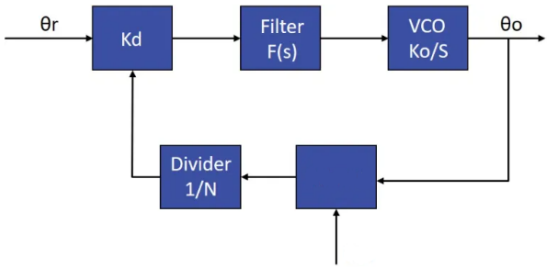
Watchdogs

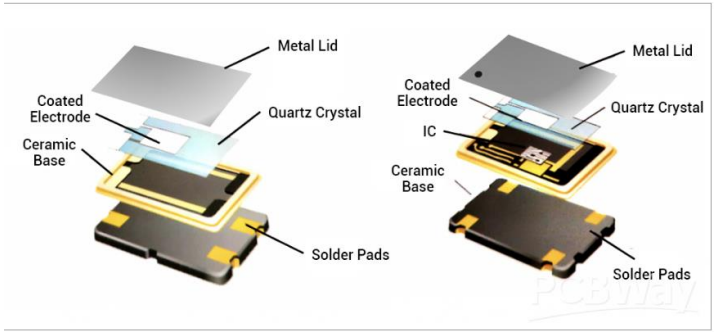
Protocoles de communication

UNIVERSITÉ LAVAL

Element	Transfer Function	Remarks
VCO	K_0/S	K_0 is tho slope of the oscillator frequency to voltage characteristic in rads per sec. per volt.
MIXER	1	Change of output phase equals change of input phase.
DIVIDER	1/N	N is division ratio.
PHASE DETECTOR	K_d	K_d is slope of phase detector voltage to phase characteristic In volts/radians.
FILTER	F(s)	Transfer function depends on the type of filter used.

Table 1: Transfer functions of phase locked loop components.





Horloge

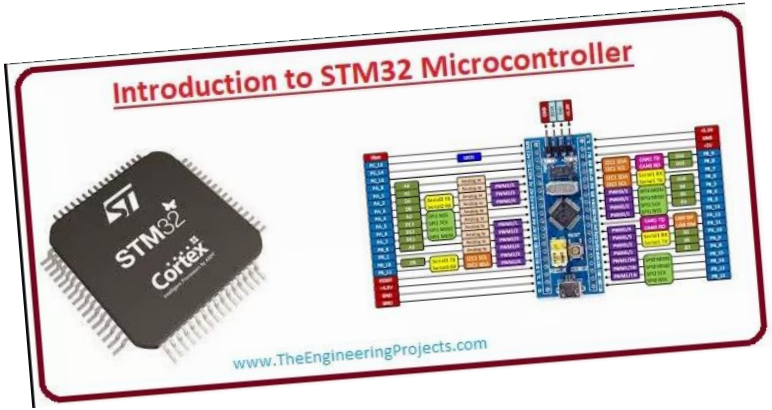
Parce que non, les horloges ne se valent pas toutes…

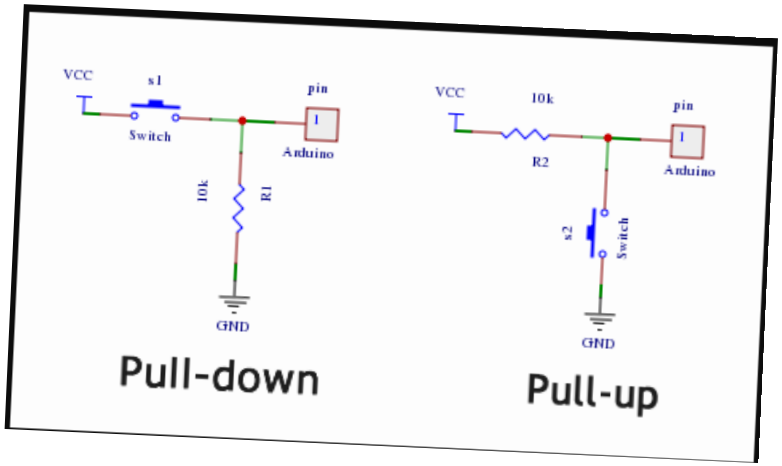
- HSI (High-Speed Internal) & HSE (High-Speed External)

Précision HSI < HSE, compatibilité USB, Ethernet, et autres

- PLL (Phase-Locked Loop)

Multiplicateur/diviseur pour atteindre une fréquence désirée pour les systèmes





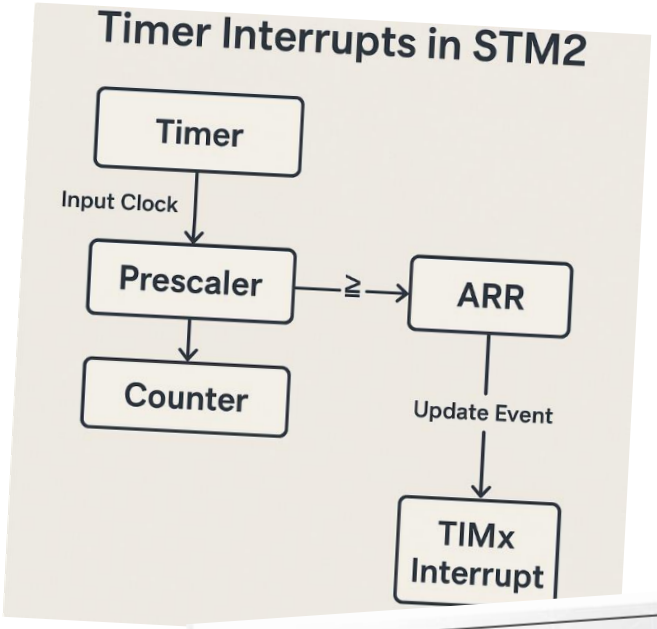
GPIOs

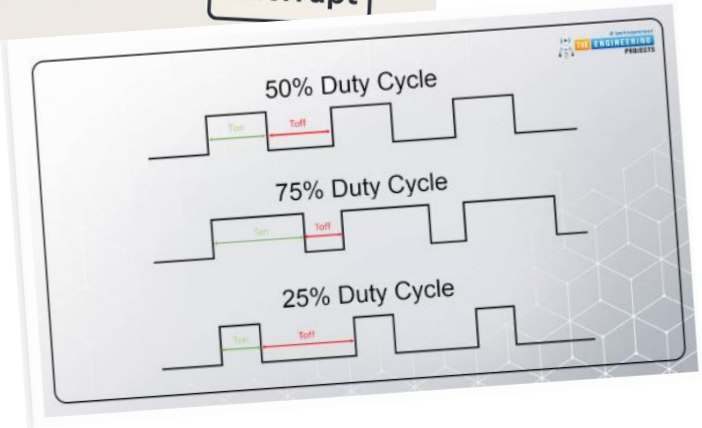
N'utilise pas le ESP8266, les GPIOs sont un mensonge …

(*General-Purpose Input/Output*)
Entrées/Sorties physiques permettant de communiquer et d'interfacer avec l'extérieur

Mode : Input, Output, Alternate Function, Analog

Options internes : Pull-up, pull-down, vitesse, push-pull





TIMER & PWM

Parce que « attendre un peu » ne suffit pas en embarqué… On n'attend que Nathan

- Timers matériels (littéralement des compteurs)

Compteurs 16/32 bits autonomes

Indépendants du MCU

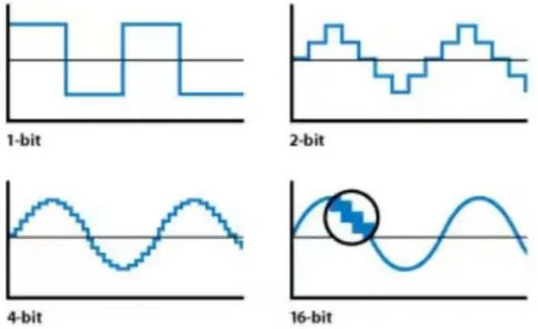
Alimentés par les horloges internes APB/PLL

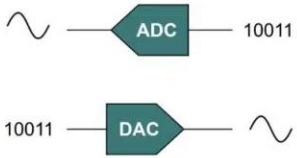
- PWM

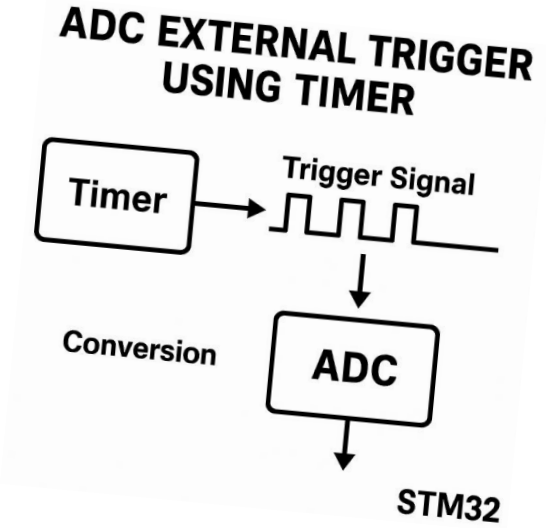
« Pulse Width Modulation » généré par des Timers

Signal carré périodique selon le rapport cyclique (duty cycle)

Permet d'imiter un signal analogique avec un signal numérique







ADC / DAC

Le monde réel n'est pas simplement fait de 0 et de 1 …

« Analogic-to-Digital »

Sert à convertir une tension analogique en valeur numérique (l'inverse pour le DAC)

Résolution 10/12/16 bits et plus

Plage entre 0-3.3/5V

Plusieurs modes disponibles et souvent 1 ou 2 ADCs par MCU, mais multiplexés sur plusieurs lignes/channels

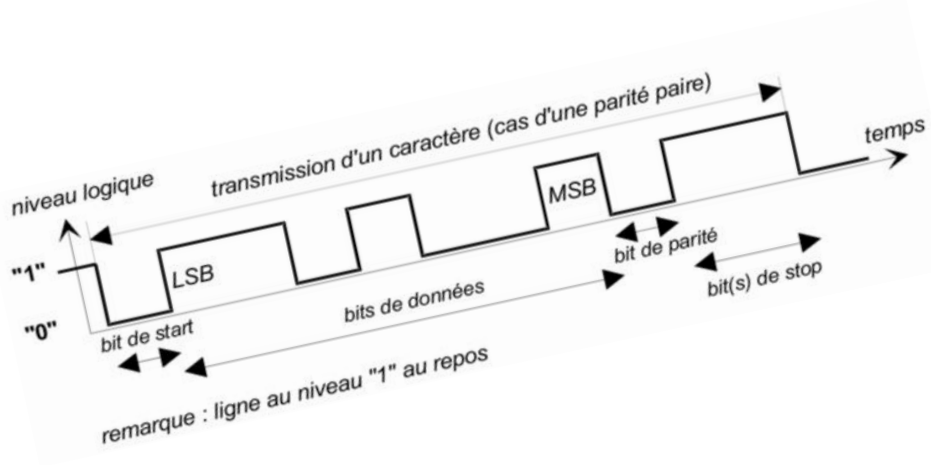


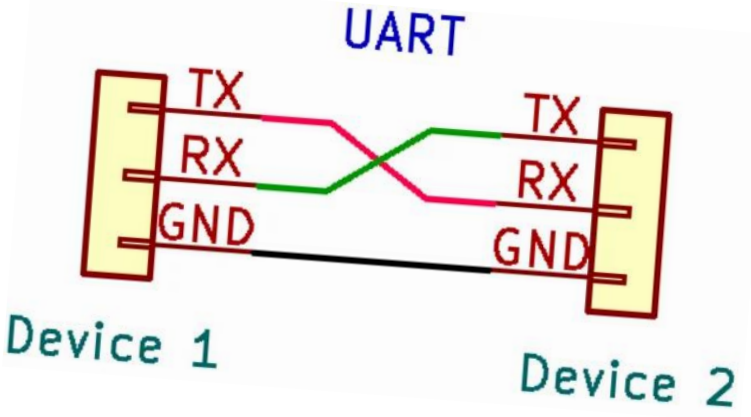
Protocoles de communication

Pour que votre MCU arrête de parler à un mur …

Chaque capteur à sa manière de communiquer l'information qu'il acquiert. Donc, on se doit d'avoir des interfaces sur notre MCU qui nous permettent de traduire la langue communiquée avec le bon traducteur

- Analogique
- UART/USART
- I2C
- SPI
- CAN
- USB/Ethernet





UART / USART

Parce que parler série, c'est le minimum syndical …

Protocole série asynchrone (UART) ou synchrone (USART) full-duplex

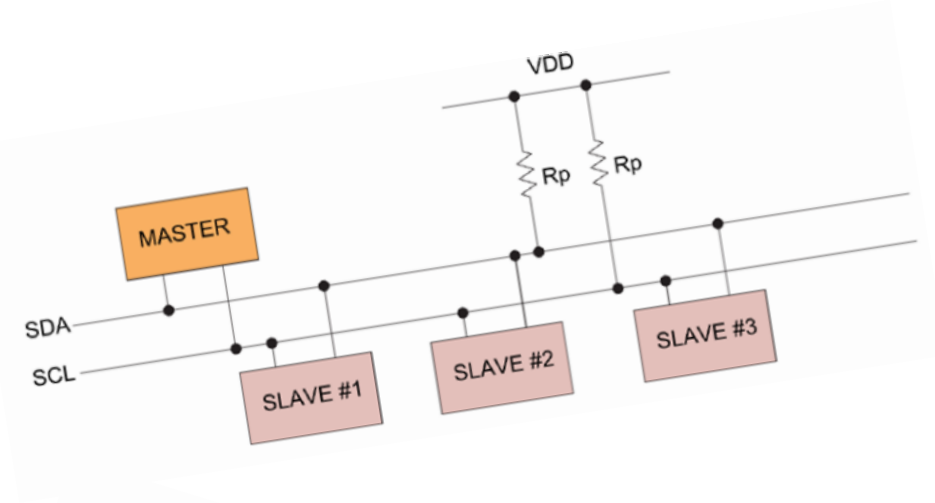
Simple, très populaire et jusqu'à 1 mètre de communication

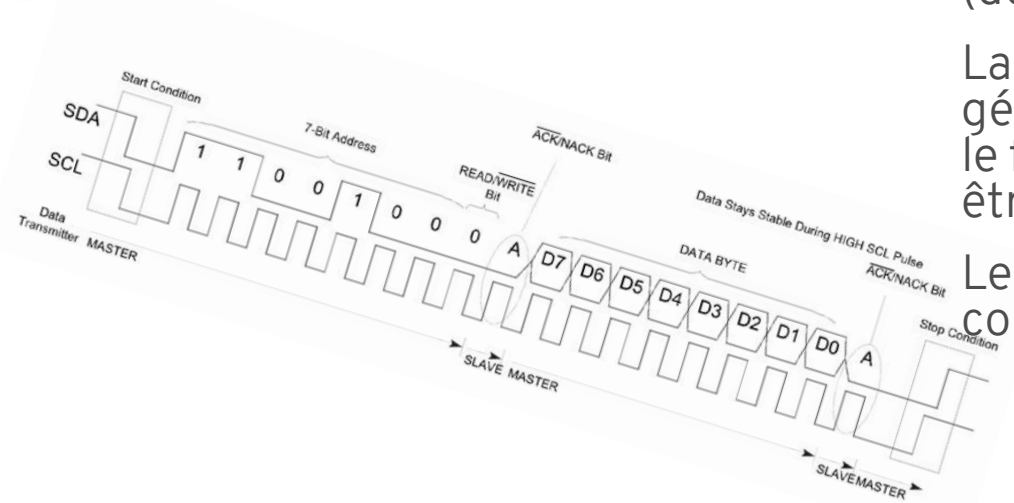
La transmission se fait au minimum avec deux fils : TX (transmission) / RX (réception)

La vitesse dépend du module (quelques Mbits/s au maximum), mais généralement de 9600bps

Si asynchrone, baudrate partagé (connu) et pas la possibilité d'avoir plusieurs esclaves. Le protocole fonctionne via une trame connue (voir image)

*Il faut inverser TX/RX entre les modules





I2C

Parce que tout gérer avec seulement 2 fils, c'est le top …

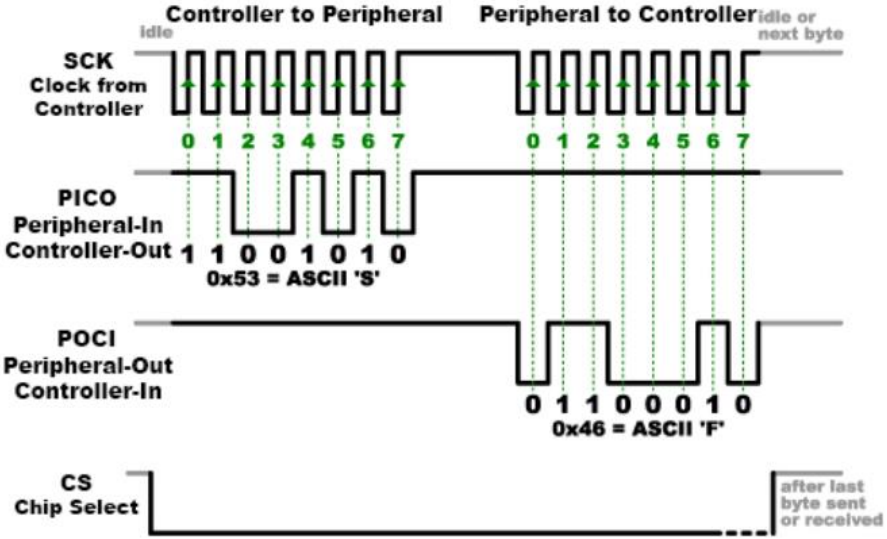
Protocole sur bus synchrone half-duplex

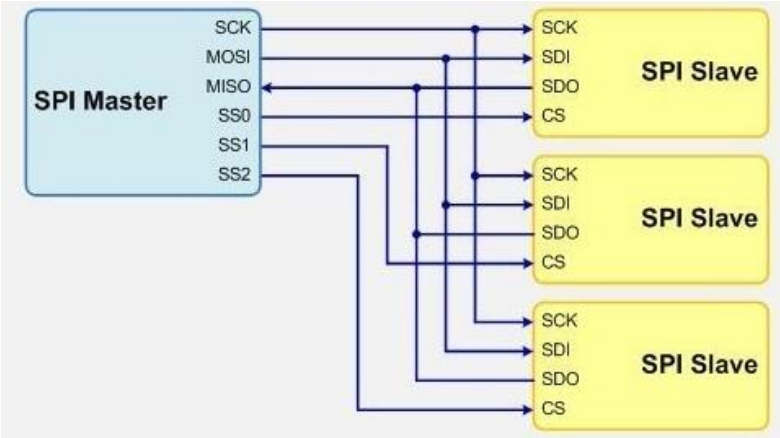
Architecture maître-esclaves adressée (127)

La transmission se fait avec deux fils : SDA (données) et SCL (horloge)

La vitesse dépend du module (1MHz), mais généralement de 400kHz. Cela limite, par le fait même, la distance maximale pouvant être parcourue par le signal

Le protocole fonctionne via une trame connue (voir image)





SPI

Quand t'es déjà en retard le matin, tu peux te permettre de rouler plus vite (ou pas) ...

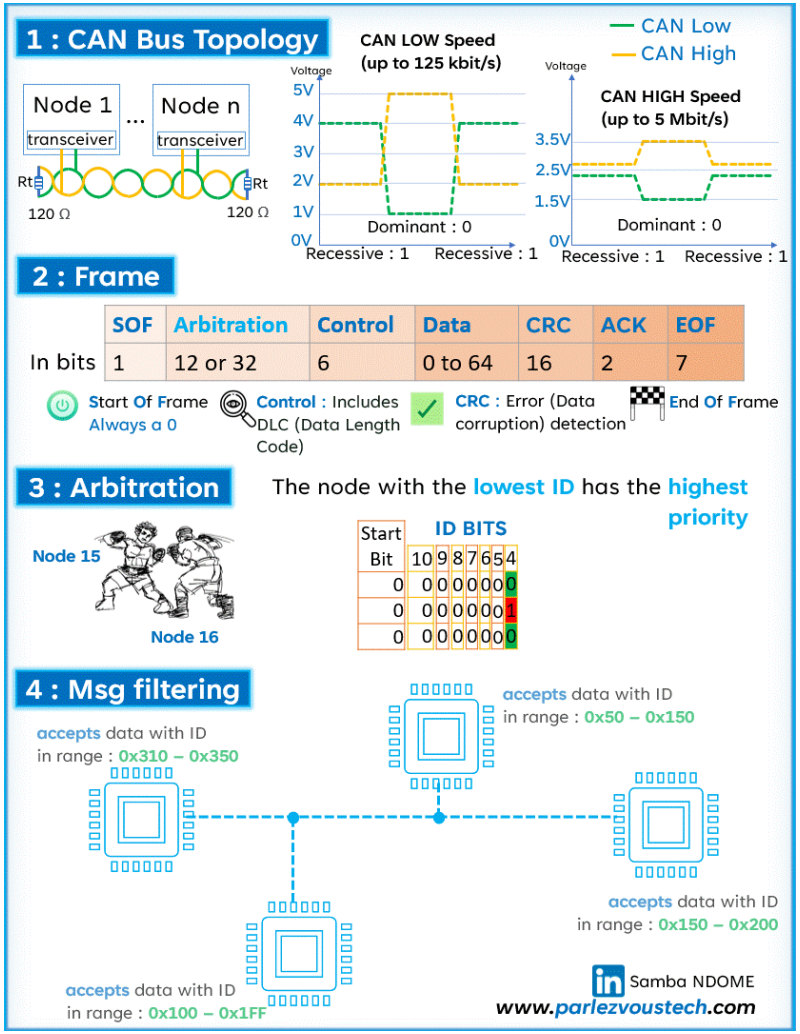
Communication synchrone full-duplex

Architecture maître-esclaves (CS)

La transmission se fait avec 4 fils : MOSI (Master Out – Slave In), MISO (inverse), SCK (Horloge) et CS (Chip select - multiple)

Très haute vitesse configurable et dépendant de l'horloge (50 Mbits/s). Cela limite, par le fait même, la distance maximale pouvant être parcourue par le signal

Le protocole communique selon le CS utilisé (voir image)



CAN

C'est comme faire une ride de char des années 80, du Québec jusqu'en Gaspésie ...

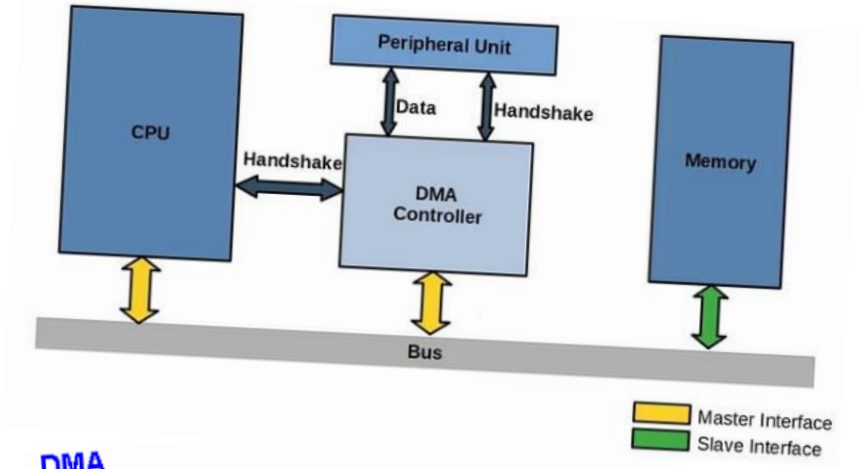
Communication différentielle half-duplex

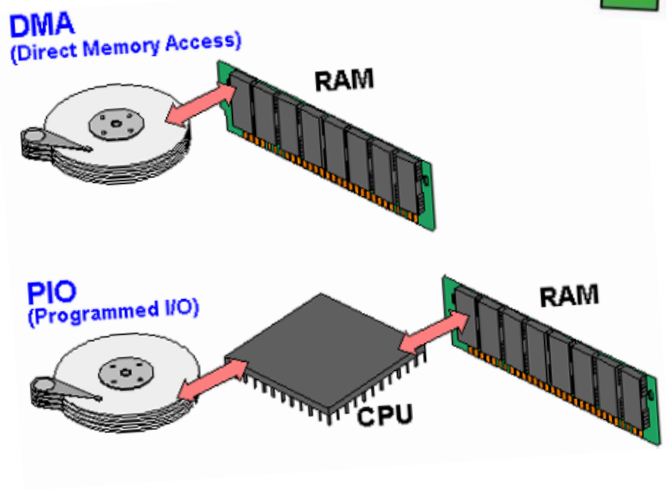
Possibilité d'avoir maître-esclaves avec priorité

La transmission se fait avec 2 fils : CANL (Low) et CANH (High)

Haute vitesse configurable et la distance n'est plus un problème grâce à la ligne différentielle à haut potentiel, le tout dans une solution ultra robuste au bruit

Nécessite des transceivers externes pour traduire le message de la ligne différentielle (±15V à du 3.3/5V)





DMA

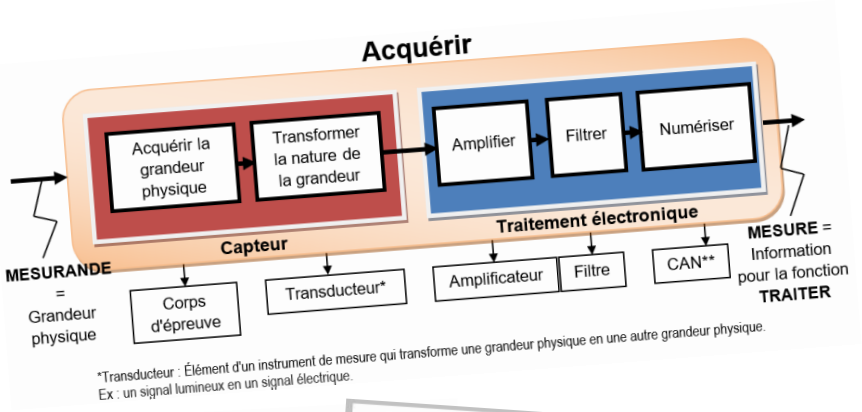
Le stagiaire que tous les ingénieurs veulent dans leur équipe (la job de RH)

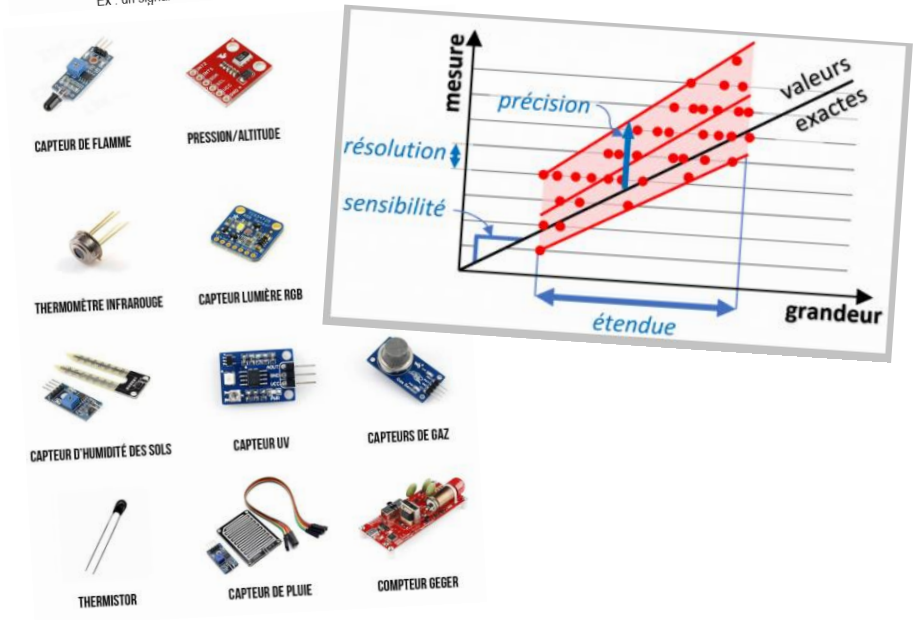
« Direct Memory Access »

Permet aux périphériques de transférer des données directement en mémoire sans intervention du MCU. Cela réduit la charge du MCU

Le module est conçu pour faire du transfert de données, donc il est aussi plus rapide que le MCU pour faire cela

Peut être pairé avec plusieurs modules intégrés au MCU : ADC, UART, SPI, I2C, TIMER, etc. (la majorité de ce qu'on a vu aujourd'hui peut être pairée avec)





Caractéristiques des capteurs

Parce que mesurer ne se résume pas juste à lire une valeur …

Chaque capteur possède ses propres limites et spécifications : plage de mesures, résolution, précision, taux d'échantillonnage, temps de réponse, consommation, etc.

Les capteurs numériques nécessitent une configuration via registres : sensibilité, fréquence d'échantillonnage, mode low-power, filtres internes… tout se règle dans les registres internes du capteur.

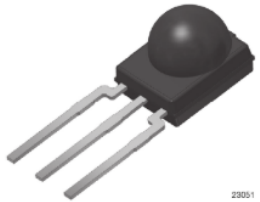
La datasheet est indispensable pour bien configurer et lire le capteur : description des registres, caractéristiques électriques, formats des données, contraintes, exemples d'utilisation.



VISHAY
www.vishay.com

TSSP930..
Vishay Semiconductors

IR Sensor Module for Reflective Sensor, Light Barrier, and Fast Proximity Applications



FEATURES

- Up to 2 m for presence and proximity sensing
- Up to 20 m distance for light curtain application
- Uses continuous AC signal or burst pattern of infrared light
- Small sensitivity scattering range
- PIN diode and sensor IC in one package
- Low supply current
- Shielding against EMI
- Visible light is suppressed by IR filter
- Insensitive to supply voltage ripple and noise
- Supply voltage: 2.0 V to 3.6 V
- Material categorization: for definitions of compliance please see www.vishay.com/doc?99912

Pb-free
e3
RoHS COMPLIANT
HALOGEN FREE GREEN (5-2008)

ADDITIONAL RESOURCES

3D Models

MECHANICAL DATA

Pinning:
1 = OUT, 2 = GND, 3 = V_S

APPLICATIONS

- Reflective sensors for hand dryers, towel or soap dispensers, water faucets, toilet flush
- Vending machine fall detection
- Security and pet gates
- Person or object vicinity switch
- Fast proximity sensors for toys, robotics, drones, and other consumer and industrial uses

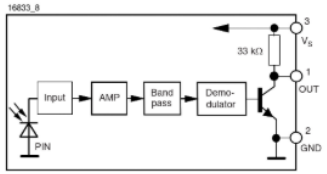
DESCRIPTION

The TSSP930.. device is the latest generation of compact infrared detector module for presence, proximity, or light curtain applications. They provide an active low output in response to infrared bursts at 940 nm. The frequency of the burst should correspond to the carrier frequency shown in the parts table.

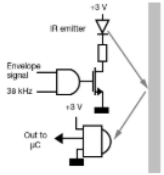
This component has not been qualified according to automotive specifications.

PARTS TABLE		
Carrier frequency	38 kHz	TSSP93038
	56 kHz	TSSP93056
Package		Minimold
Pinning		1 = OUT, 2 = GND, 3 = V_S
Dimensions (mm)		5.4 W x 6.35 H x 4.9 D
Mounting		Leaded
Application		Presence sensors, fast proximity sensors

BLOCK DIAGRAM



PRESENCE SENSING



Comment choisir un capteur ?

Parce qu'il n'existe pas de capteur universel (j'espère que je ne vous apprends rien ici ...)

Il faut d'abord définir son besoin réel : la plage de mesures, précision, vitesse d'acquisition, bruit, environnement dans lequel je vais mesurer, etc...

Ensuite, tout va dépendre de la compatibilité avec le système : le type de communication, tension d'alimentation, consommation, disponibilité matérielle

Finalement, avant de faire un choix, assure-toi d'avoir le support/des informations claires (genre Amazon, oublie ça). Parfois, choisir la solution populaire nous évite plusieurs problèmes futurs

UNIVERSITÉ LAVAL

Lecture d'un capteur

Y'a des étapes entre prendre une mesure et avoir une vraie valeur …

Ben, premièrement, faut lire la sortie … Si c'est analogique, on y va avec un ADC, sinon, si c'est numérique, on utilise le bon protocole de communication

Une fois que cela est fait, la datasheet nous détaille généralement tout ce dont on a besoin pour la suite : calibration, équation pour convertir la sortie en valeur réelle, mise à l'échelle, filtrage, compensation, State machine, etc.

Finalement, on peut vérifier les statuts du capteur : flags, saturation, validité des données. Une mauvaise lecture mène généralement à une mauvaise valeur interprétée

3.4. Measurement flow

The BMP390 measurement period consists of a temperature and pressure measurement with selectable oversampling. After the measurement period, the data are passed through an optional IIR filter, which removes short-term fluctuations in pressure (e.g. caused by slamming a door). The flow is depicted in the diagram below.

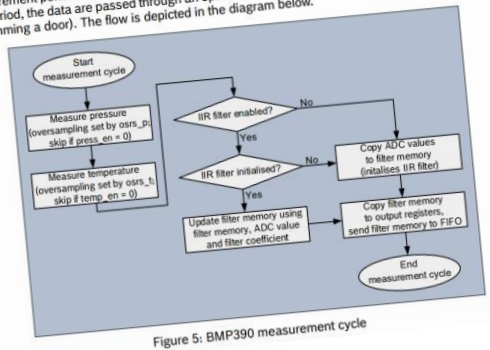


Figure 5: BMP390 measurement cycle

Table 10: Recommended filter settings based on use cases

Use case	Mode	Over-sampling setting	osrs_p	osrs_t	IIR filter coeff. (see 4.3.21)	I_{DD} [µA] (see 3.8)	ODR [Hz] (see 3.4.1)	RMS Noise [cm] (see 3.4.4)
handheld device low-power (e.g. Android)	Normal	High resolution	x8	×1	2	145	12.5	11
handheld device dynamic (e.g. Android)	Normal	Standard resolution	×4	×1	4	310	50	10
Weather monitoring (lowest power),	Forced	Ultra low power	×1	×1	Off	4	1/60	55
Drop detection	Normal	Low power	×2	×1	Off	358	100	36
Indoor navigation	Normal	Ultra high resolution	×16	×2	4	560	25	5
Drone	Normal	Standard Resolution	x8	x1	2	570	50	11
Indoor localization	Normal	Ultra low power	x1	x1	4	-	1	-

Questions ?

Pause de 10min

UNIVERSITÉ LAVAL

PROGRAMMATION EMBARQUÉE EN C

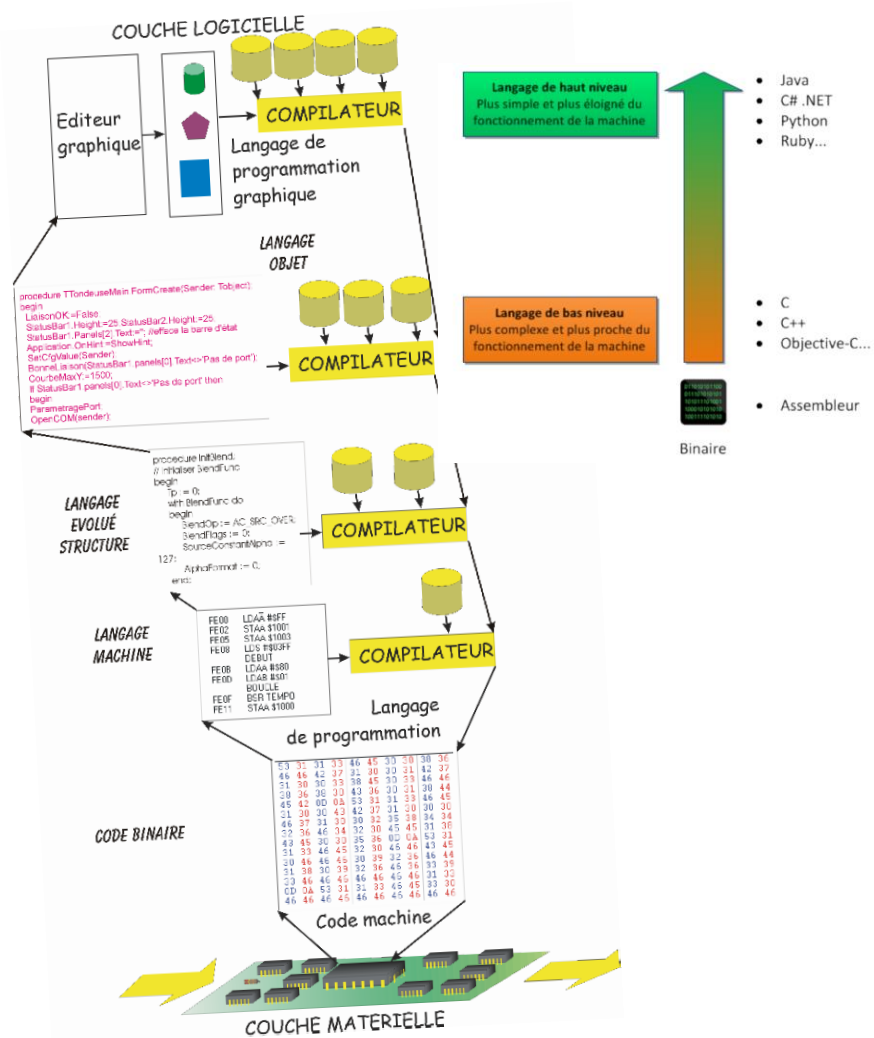




Programmation en C

Comment est-ce qu'on traduit ça en C?

- C'est quoi le C a priori, et pourquoi ça plutôt que Python ?
- STM32CubeIDE
- main.c (initialisation, boucle principale)
- Drivers
- Structure (configuration)
- De la pensée à la fonction
- Vérifications



C'est quoi le C

Le langage qui existe depuis 50 ans et qui refuse de mourir ...

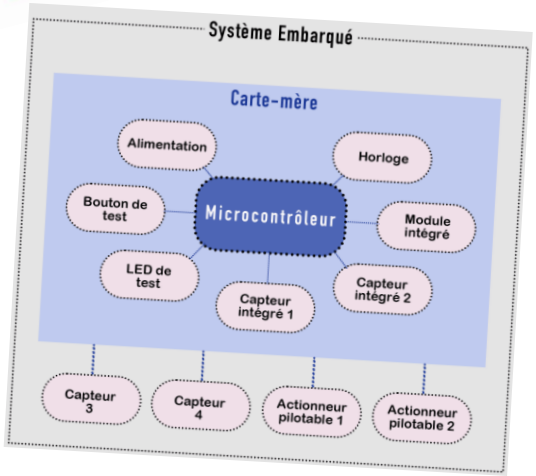
Plusieurs niveaux de langages en programmation, le C est bas-niveau

Contrôle précis et rapide sur le matériel (mémoire)

Utilisé dans plusieurs systèmes actuels comme Windows (Unix), micro-ondes, systèmes d'alarme, la laveuse que tu n'utilises pas assez aussi

Langage compilé : un compilateur traduit le code en langage machine (Assembleur->binaire)





Pourquoi utiliser le C en embarqué

Parce que du Python sur un microcontrôleur … ça finit vite par brûler

Accès direct au matériel et contrôle de la mémoire de façon précise et performante (temps réel)

Sa popularité participe aussi à son utilisation (Rust vs C)

Fonctionne avec quelques Ko de RAM/Flash, contrairement aux langages plus haut niveau comme le Python

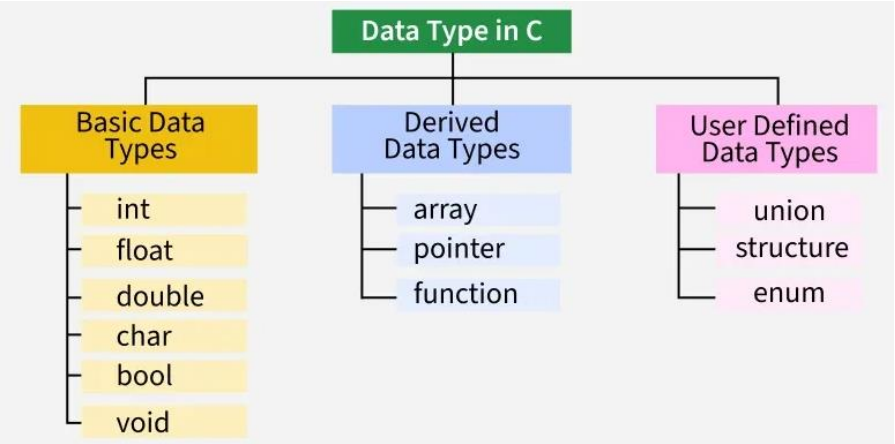
En embarqué, on programme des modules qui sont limités en mémoire et en performance dû à leur taille/consommation/vitesse, donc il est essentiel d'utiliser un langage performant et qui minimise son empreinte mémoire



Bases pour programmer en C

Parce que pour être bon en embarqué, faut être bon en C …

- Variables, Types et Opérations
- Structures & Fonctions
- Contrôles conditionnels

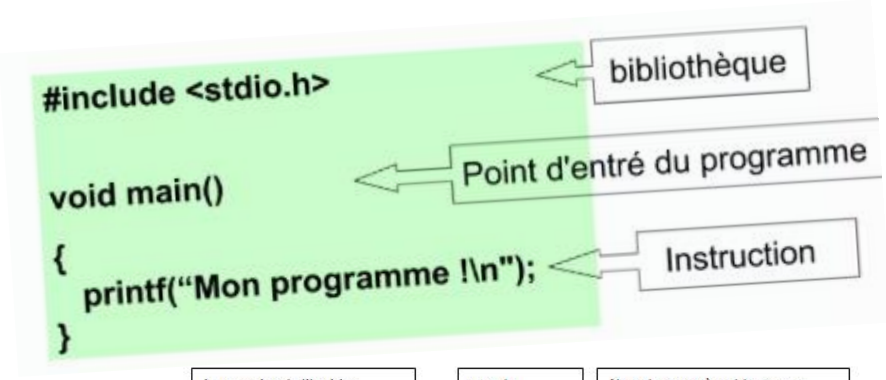


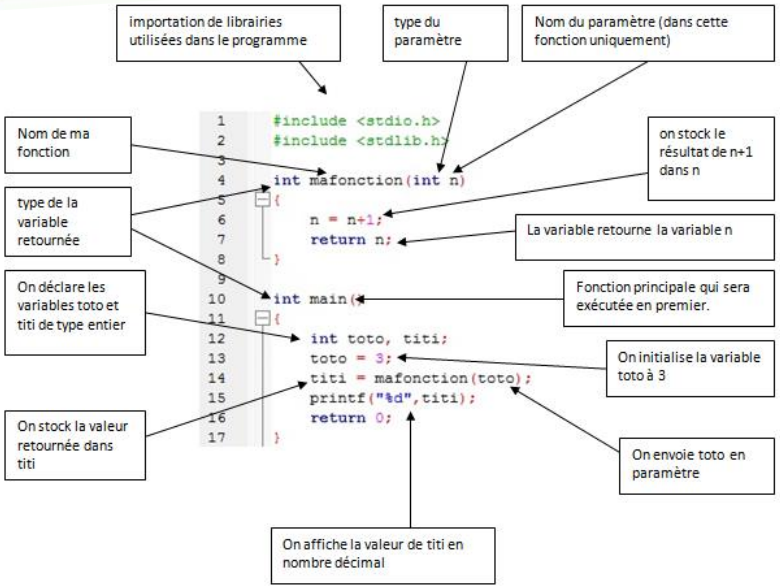
Variables, Types et Opérations

J'ai rien à dire sur ça, c'est juste la base

- Types de base
- Tableaux : séquence de valeurs de mêmes types
- Pointeurs : adresse mémoire (la vie)
- Opérateur : ceux de base et surtout les déplacements de bits

Operator	Description	Example (C=10, D=20)
+	This is the additive operator that calculates the sum of the two operands.	C + D = 30
–	It subtracts the value of the right-hand operand from the left side operand.	C – D = -10
*	This operator multiplies the two operands.	C * D = 200
/	Divides the operand in the numerator by the operand in the denominator.	D / C= 2
%	The modulus operator gives out the remainder for an integer division.	D % C = 0
++	The increment operator increases the integer value by one.	C++ = 11
--	The decrement operator decreases the integer value by one.	C-- = 9

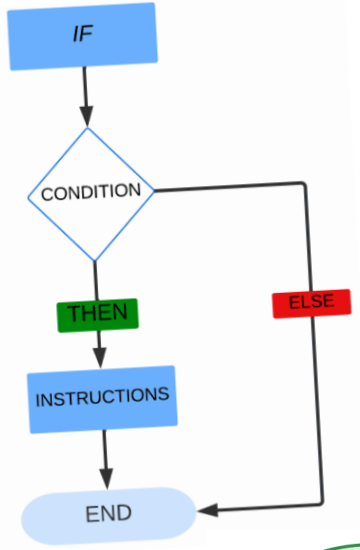


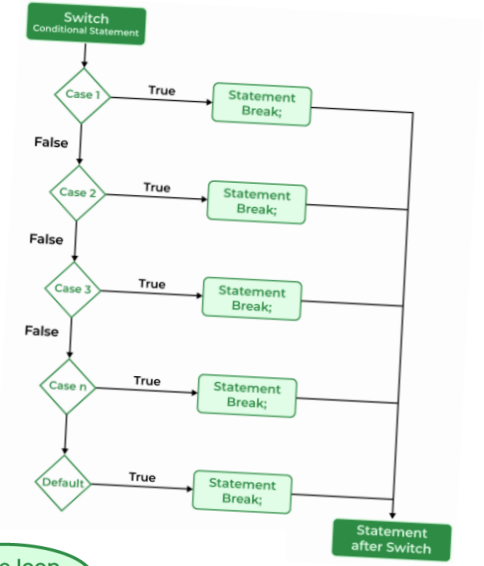


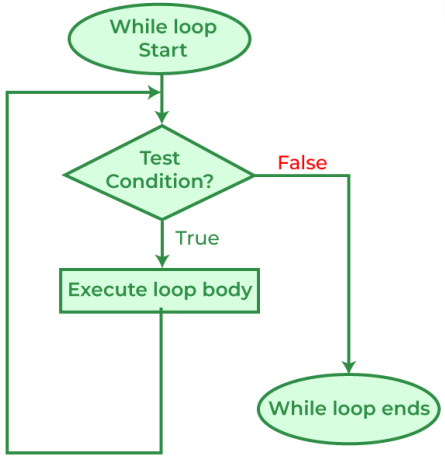
Structures & Fonctions

Même chose pour cette slide-là, fait juste écouter et ça devrait être correcte ☺

- Modularité : header/source file
- Fonctions : Servent à éviter la redondance et permettent de retourner une sortie selon une/plusieurs entrées (paramètres)
- Les fameux nombres magiques







Contrôles conditionnels

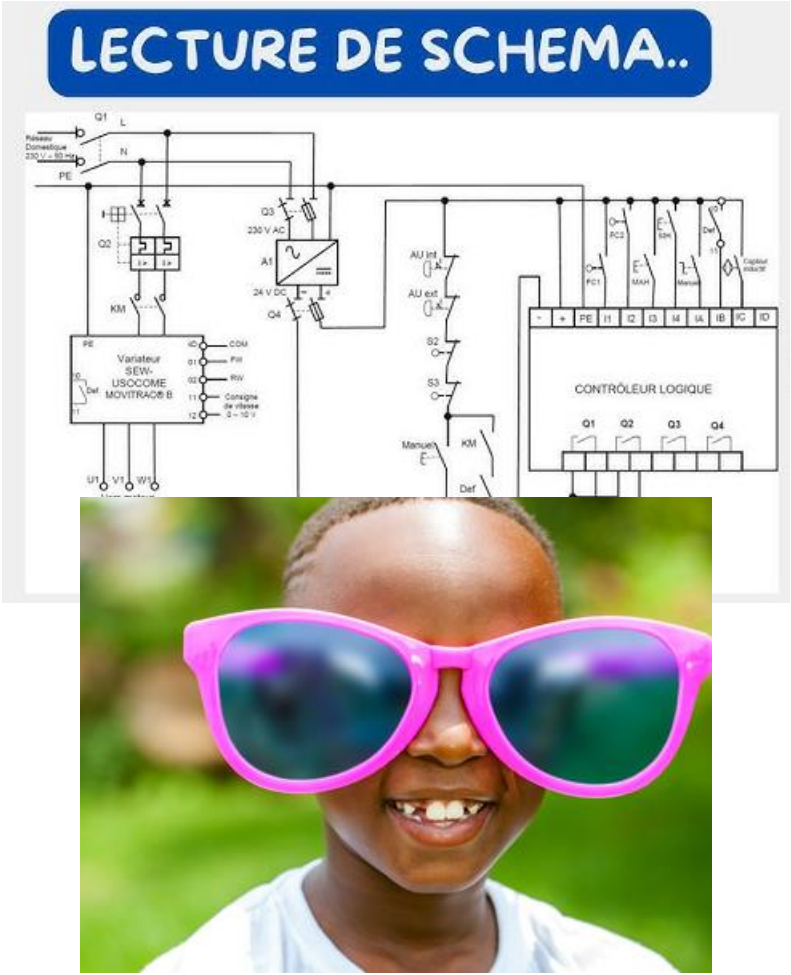
Tu peux apprendre plus if tu es intéressé, else retourne à 0

- if, else, else if
- For, while, do while()
- Switch-case
- State machine

TON **COURS** DE **C++** L'AN PROCHAIN VA T'AIDER POUR SÛR !!!

(J'ESPÈRE EN TOUT CAS)

UNIVERSITÉ LAVAL



Bases de la programmation embarquée

Si seulement c'était juste de la programmation, ça serait facile …

Tu dois pouvoir comprendre le matériel pour le programmer (c'est juste logique). Ça inclus où aller chercher l'information (registres, datasheet, ton prof un peu trop arrogant), comment y avoir accès (protocole de communication, entrées/sorties, fonctionnement des systèmes, configurations) et pourquoi tu en as besoin (ex. : Le petit Thomas aimerait vraiment mesurer l'humidité extérieure pour savoir quand sortir son parapluie).



Structure en embarqué au GAUL

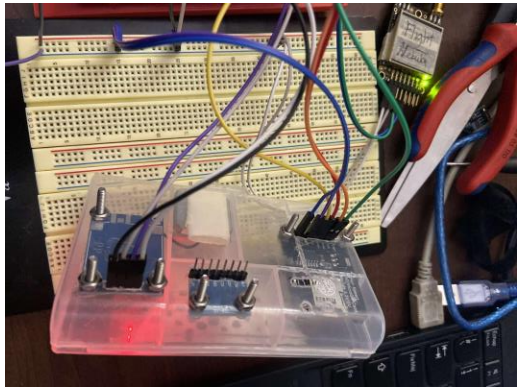
Allez Samuel! J'ai tellement hâte de savoir comment vous faites au GAUL pour être les meilleur(e)s …

- STM32CubeIDE (Je te montre ça maintenant !!!!!)
- main.c (initialisation, boucle principale)
- Drivers
- Structure (configuration et hiérarchisation du programme)
- De la pensée à la fonction (Je ne pense pas, donc j'existe ..?)
- Vérifications

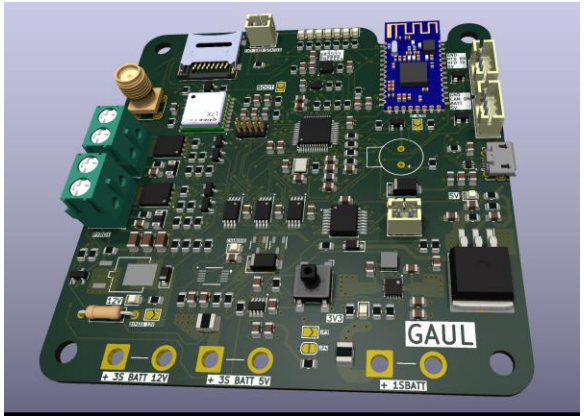
ORDINATEUR DE BORD

(SURVOL)

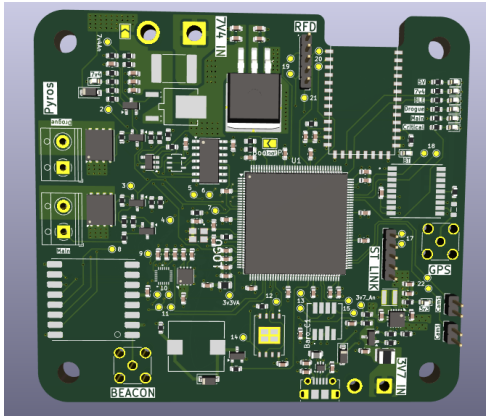
Arduino 2022-2023



ODB1 2023-2024



ODB2 2024-2025



ODBlix 2025-2026



Ordinateur de bord

2022 à aujourd'hui…

Objectif : englober tous les requis avioniques sur un PCB robuste et équipé de capteurs précis.

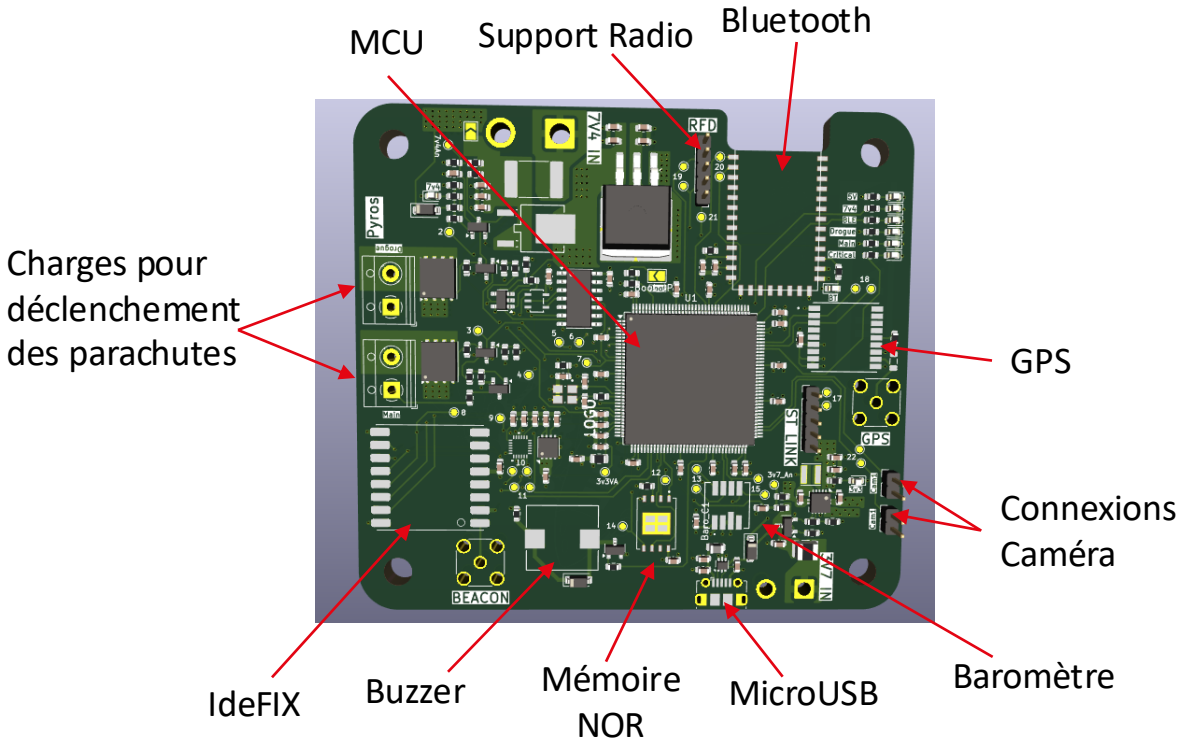
Première itération:

GPS, IMU 6-axes, Baromètre, SD, Radio, Bluetooth, Pyros(2x), Buzzer, Support Caméras(2x)

Deuxième itération:

GPS, IMU 9-axes, Baromètre, Mémoire NOR, Radio, Bluetooth, Buzzer, Pyros(2x), IdeFIX, Support Caméras(2x)

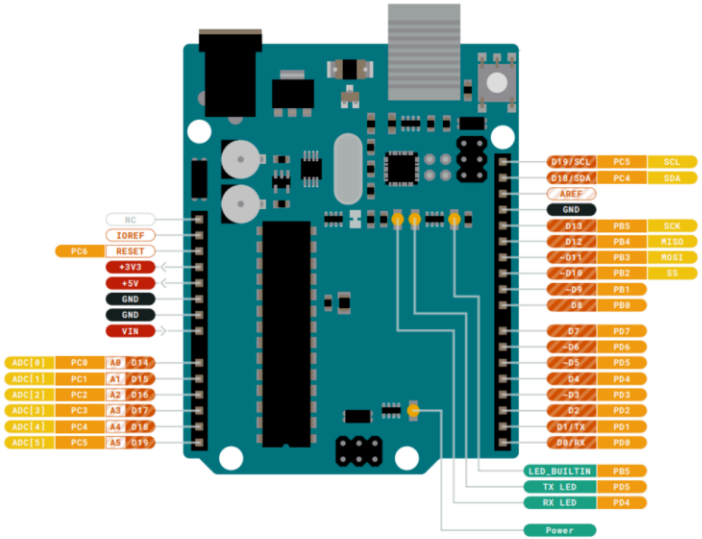
Ordinateur de bord



Questions ?

MINI PROJET

DE LA THÉORIE À LA PRATIQUE



Les modules à programmer

Comme nous sommes pauvres, on va gosser sur un Arduino pour simplifier tout ça ...

Arduino Uno R3

- ATmega328p : 16MHz,
- Nombre d'IOs (14 Digitals, 6 Analogs, 6 PWMs)
- UART/I2C/SPI
- Débogage USB
- Alimentation de fonctionnement : 5V
- Courant par IO max : 20mA

https://docs.arduino.cc/hardware/uno-rev3/#tech-specs

Les modules à programmer

" " " " " " " " " " " " " " " " " " " , on va gosser sur un Arduino pis … une LED …

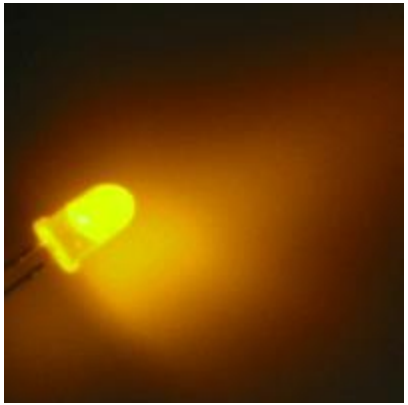
Couleurs	Tension de seuil OU Vf	If (mA)	Longueur d' onde
Rouge	1,6 V à 2 V	6à20	650 à 660 nm
Jaune	1,8 V à 2 V	6à20	565 à 570 nm
Vert	1,8 V à 2 V	6à20	585 à 590 nm
Bleu	2,7 V à 3,2 V	6à20	470 nm
blanc	3,5 v à 3,8 v	30	

LED rouge

Comme on doit limiter le courant par IO à 20mA et que ça prend au moins 2V pour l'allumer, on doit ajouter une résistance :

R > V/Imax > 5V / 20mA > 250 ohms





Les modules à programmer

"""""""""""""""", on va gosser sur un Arduino pis une LED … pis on va la faire clignoter … (ooooooohhh, c'est tellement incroyable ça)

Ce qu'on va faire c'est créer des routines de clignotement pour notre LED

On a besoin de quoi pour faire ça au juste de façon structurée et réutilisable ? :

- Une broche digitale et configurée (0=0V, 1=5V)
- Un struct pour la configuration de notre LED
- Un enum pour stocker nos routines de clignotement
- Des fonctions pour faire le changement de routine
- Une boucle pour répéter tout ça à l'infinie parce que c'est cool…

Passons au code maintenant que ce qu'on veut est défini!!! (Regarde, je vais faire du code avec toi)

*Techniquement, on devrait utiliser un PWM pour ne pas user la LED, mais bon. Pour aujourd'hui, c'est bon. Je te donne l'autorisation de ne pas faire ça ;)

```
1  // ------------------------------------------------------------
2  // INCLUDES : Bibliothèques nécessaires
3  // Explication : Inclusion des bibliothèques nécessaires pour le projet
4  // ------------------------------------------------------------
5  //#include <Arduino.h>
```

```
// ------------------------------------------------------------
//  ENUM : Différentes routines de clignotement
//  Explication : Définit les différents modes de clignotement
//  utilisés par la LED. Ces valeurs sont utilisées pour choisir
//  le comportement dans les fonctions de mise à jour.
// ------------------------------------------------------------
enum BlinkMode {
  BLINK_SLOW,
  BLINK_FAST,
  BLINK_DOUBLE,
  BLINK_OFF
};
```

```
20  // ------------------------------------------------------------
21  //  STRUCT : Contient la configuration d'une LED (instance)
22  //  Explication : Regroupe les paramètres nécessaires pour
23  //  piloter une LED : broche, mode, période de clignotement
24  //  et horodatage de la dernière action.
25  // ------------------------------------------------------------
26  typedef struct {
27    int pin;                // Broche utilisée
28    BlinkMode mode;         // Mode de clignotement
29    unsigned long period;   // Période de clignotement calculée selon le mode (ms)
30    unsigned long lastTime; // Dernier horodatage d'action (ms)
31  } LedConfig;
```

```
33  // ------------------------------------------------------
34  //  PROTOTYPES DES FONCTIONS
35  //  Explication : Déclaration des fonctions manipulant la LED.
36  //  Elles prennent une référence vers LedConfig pour modifier
37  //  l'état de la configuration depuis l'appelant. Comme le C est un langage procédural,
38  //  on simule un passage par référence par pointeur (*).
39  // ------------------------------------------------------
40  void setBlinkMode(LedConfig *led, BlinkMode mode);
41  void updateLed(LedConfig *led);
42  void blinkRoutine(LedConfig *led);
```

```
// ------------------------------------------------------
//  CONFIGURATION
//  Explication : Instance globale représentant la LED utilisée (l'objet LED).
//  Ici, on initialise la LED sur la broche 13 et le mode clignotement lent (1 seconde) par défaut.
// ------------------------------------------------------
LedConfig led = {
  13,                 // pin D13 (numérique)
  BLINK_SLOW,         // mode par défaut
  1000,               // période initiale (1 seconde)
  0                   // horodatage initialisé à 0
};

```

```
56  void setup() {
57    // ------------------------------------------------------
58    //  SETUP (démarrage) : Initialisation du matériel
59    //  Explication : Fonction appelée une seule fois au démarrage/reset
60    //  pour configurer la communication série et la broche LED.
61    // ------------------------------------------------------
62    Serial.begin(9600);               // Initialise le port série à 9600 bauds
63    pinMode(led.pin, OUTPUT);         // Configure la broche de la LED en sortie
64    digitalWrite(led.pin, LOW);       // Assure que la LED est éteinte au départ
65    setBlinkMode(&led, BLINK_OFF);    // Définit le mode par défaut (mise à jour de period)
66    led.lastTime = millis();          // Initialise le timer
67  }
```

```
// ------------------------------------------------------
//  BOUCLE PRINCIPALE (loop)
//  Explication : Appelée en continu et de facon procédurale (ligne par ligne). On change le mode de la LED
//  selon le temps écoulé puis met à jour l'instance LED.
// ------------------------------------------------------
void loop() {
  // Changement de mode avec conditions
  // Important : on utilise millis() pour ne pas bloquer la boucle principale, sinon le programme serait figé pendant ce temps.
  unsigned long now = millis();
  if (now < 5000) {
    setBlinkMode(&led, BLINK_SLOW);    // De 0s à 5s -> clignotement lent
  }
  else if (now >= 5000 && now < 10000) {
    setBlinkMode(&led, BLINK_FAST);    // De 5s à 10s -> clignotement rapide
  }
  else if (now >= 10000 && now < 15000) {
    setBlinkMode(&led, BLINK_DOUBLE);  // De 10s à 15s -> double clignotement
  }
  else if (now >= 15000) {
    setBlinkMode(&led, BLINK_OFF);     // Après 15s -> LED éteinte
  }

  // Mise à jour de la LED selon la routine (non bloquante sauf BLINK_DOUBLE)
  updateLed(&led);
}
```

```
// ---------------------------------------------------------
//  FONCTION : Changer de mode de clignotement
//  Explication : Modifie la configuration (mode et période)
//  et remet à jour lastTime pour synchroniser le prochain cycle.
// ---------------------------------------------------------
void setBlinkMode(LedConfig *led, BlinkMode mode) {
  // Si le mode change, on met à jour la période, sinon on ne fait rien (limite les recalculs inutiles)
  if (led->mode != mode) {
    led->mode = mode;                   // Met à jour le mode
    Serial.print("Changement de mode : ");
    Serial.println(mode);

    // Détermine la période selon le mode
    switch (mode) {
      case BLINK_SLOW:
        led->period = 1000;             // 1 seconde
        break;
      case BLINK_FAST:
        led->period = 200;             // 0.2 seconde
        break;
      case BLINK_DOUBLE:
        led->period = 150;             // utilisé pour les bursts (petits délais)
        break;
      case BLINK_OFF:
        digitalWrite(led->pin, LOW);   // Éteint immédiatement la LED
        led->period = 0;
        break;
      default:                         // Sécurité : ne rien faire
        break;
    }

    // Important : on réinitialise le timer pour éviter des déclenchements immédiats
    led->lastTime = millis();
  }
}
```

```
130 // ------------------------------------------------------------
131 //  FONCTION : Mise à jour en fonction du mode
132 //  Explication : Vérifie si la période est écoulée et déclenche
133 //  la routine de clignotement. Ne fait rien si BLINK_OFF.
134 // ------------------------------------------------------------
135 void updateLed(LedConfig *led) {
136   // BLINK_OFF -> pas d'action (limite les lectures inutiles)
137   if (led->mode == BLINK_OFF || led->period == 0) return;
138
139   unsigned long current = millis();
140
141   // Si le temps écoulé dépasse la période, on exécute la routine
142   if (current - led->lastTime >= led->period) {
143     blinkRoutine(led);
144     led->lastTime = current; // Mise à jour du timestamp après action
145   }
146 }
```

```
// ------------------------------------------------------
//  FONCTION : Exécute la routine de clignotement
//  Explication : Comporte différents comportements selon le mode.
// ------------------------------------------------------
void blinkRoutine(LedConfig *led) {
  // Comportement selon le mode
  switch (led->mode) {
    case BLINK_SLOW:
      // simple ON/OFF : on inverse l'état actuel de la broche
      digitalWrite(led->pin, !digitalRead(led->pin));
      break;
    case BLINK_FAST:
      // simple ON/OFF : on inverse l'état actuel de la broche
      digitalWrite(led->pin, !digitalRead(led->pin));
      break;
    case BLINK_DOUBLE:
      // ON -> OFF -> ON -> OFF (séquence bloquante volontaire)
      // Pause bloquante pour le double clignotement (on aurait pu le faire non bloquant avec un état supplémentaire)
      digitalWrite(led->pin, HIGH);
      delay(led->period);
      digitalWrite(led->pin, LOW);
      delay(led->period);
      digitalWrite(led->pin, HIGH);
      delay(led->period);
      digitalWrite(led->pin, LOW);
      break;
    default:
      break;
  }
}
```

BRAVO! TU SAIS MAINTENANT FAIRE CLIGNOTER UNE **LED**





Liens utiles

Si jamais ça t'intéresse plus que le cours que je viens de donner …

Introduction aux projet embarqués STM32 : https://blog.embeddedexpert.io/

Tutoriel embarqué: https://deepbluembedded.com/

* Écoute pour le reste, je crois que tu connais YouTube et Google. T'as juste à taper Google dans Google et taper ta question ! Si tu vois StackOverflow, t'as trouvé la bible.

Questions ?



NOTE À MOI-MÊME VA **FERMER L'ENREGISTREMENT** DE LA RENCONTRE MAINTENANT

(TU VAS OUBLIÉ SINON)

UNIVERSITÉ LAVAL